

Bản trình chiếu: Ứng dụng của thứ tự

Long Fan

2005

Nguồn gốc: Ứng dụng của thứ tự: bản trình chiếu

IOI China candidate team papers / 2005 / Long Fan / Long Fan.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết về quan hệ thứ tự. Văn bản trích được từ slide xác nhận các mốc chính: công thức $t(i)$, vị trí $t(j) + 1$, các độ phức tạp $\mathcal{O}(N \log N)$, $\mathcal{O}(N)$, $\mathcal{O}(N^2)$, $\mathcal{O}(N \log(N + M))$, cùng hai loại lệnh Goto và NewGoto. Bản ghi chú này dùng bài viết đã dịch làm neo để phục hồi luồng slide: tìm thứ tự ẩn trong dữ liệu, biến thao tác phức tạp thành thao tác theo thứ tự, rồi dùng cấu trúc dữ liệu đơn giản để thực hiện.

Ý tưởng chung là: nhiều bài toán không khó vì công thức chuyển trạng thái phức tạp, mà khó vì ta chưa chọn đúng thứ tự xử lý. Khi thứ tự đúng xuất hiện, các phần tử “đã quyết định” và “chưa quyết định” tách nhau rõ ràng; khi đó ta có thể quét, quy hoạch động, sắp xếp tô-pô, hoặc tìm ô trống thứ k bằng cây Fenwick.

2 Tư tưởng

Thứ tự không chỉ là quan hệ lớn nhỏ trong một mảng đã sắp. Trong bài thi, thứ tự có thể là thứ tự DFS của một cây, thứ tự tô-pô của phụ thuộc, thứ tự trái-phải của các vị trí trống, hoặc thứ tự trước-sau do một thao tác đẩy gây ra. Slide nhấn mạnh việc phát hiện thứ tự này trước khi chọn kỹ thuật cài đặt.

Ví dụ quen thuộc nhất là tìm kiếm nhị phân: chỉ khi dữ liệu có tính đơn điệu, ta mới được phép loại một nửa không gian. Quy hoạch động trên DAG cũng vậy: đỉnh chỉ được tính sau khi mọi tiền nhiệm cần thiết đã được tính. Hai ví dụ chính trong slide cho thấy cùng một nguyên lý ở dạng kín đáo hơn: một bài dựng trọng số trên cây và một bài xếp hàng bằng lệnh chen.

3 Bài dựng cây

Cho một cây có n đỉnh. Mỗi đỉnh v nhận một trọng số $s(v)$, và các trọng số tạo thành một hoán vị của $1, 2, \dots, n$. Với mỗi đỉnh, đề cho $t(v)$, là số hậu duệ của v có trọng số nhỏ hơn $s(v)$. Cần dựng một cách gán trọng số thỏa mãn các giá trị $t(v)$.

Thoạt nhìn, thông tin $t(v)$ là quan hệ giữa một đỉnh và toàn bộ cây con của nó. Điểm then chốt của slide là dùng thứ tự duyệt trước DFS. Trong dãy duyệt trước, các đỉnh thuộc cây con của v nằm trong một đoạn liên tiếp ngay sau v . Nhờ vậy, khi xử lý đỉnh theo thứ tự DFS, ta có thể hiểu việc gán trọng số như việc điền đỉnh vào các ô trống biểu diễn trọng số 1 đến n .

Quy tắc là: khi gặp đỉnh i , đặt nó vào ô trống thứ $t(i) + 1$ tính từ trái sang. Các ô trống bên trái được chứa lại chính là chỗ dành cho những hậu duệ có trọng số nhỏ hơn $s(i)$. Đây là mốc $t(j) + 1$ còn đọc được trong slide. Chứng minh dựa trên quy nạp theo cây con: với một cây con bất kỳ, toàn bộ các đỉnh của nó sẽ chiếm một đoạn ô trống liên tiếp, nên những ô chứa lại trước gốc cây con cuối cùng đều được hậu duệ của nó lấp.

Sau khi nhìn ra thứ tự này, phần còn lại trở thành bài toán chuẩn: có n ô trống, mỗi lần cần tìm ô trống thứ k và đánh dấu nó đã dùng. Cây Fenwick hoặc cây đoạn lưu số ô trống trong từng đoạn, hỗ trợ tìm ô trống thứ k trong $\mathcal{O}(\log N)$. Duyệt DFS mất $\mathcal{O}(N)$, tổng thời gian là

$$\mathcal{O}(N \log N),$$

đúng với mốc độ phức tạp trên slide.

4 Goto và NewGoto

Ví dụ thứ hai là bài xếp hàng binh sĩ. Một lệnh $\text{Goto}(L, S)$ đặt binh sĩ S vào vị trí L . Nếu vị trí đã có người, người cũ bị đẩy sang phải, rồi có thể tiếp tục đẩy dây chuyền. Mô phỏng trực tiếp dễ mất $\mathcal{O}(N^2)$, cũng là một mốc còn đọc được từ slide.

Slide đổi cách nhìn bằng lệnh $\text{NewGoto}(L, S)$: thay vì đặt vào vị trí tuyệt đối L , ta đặt S vào ô trống thứ L . Với định nghĩa mới, không còn dây chuyền đẩy; vấn đề là phải tìm một thứ tự thực hiện các cặp (L, S) sao cho kết quả cuối giống dãy Goto ban đầu.

Thứ tự ấy là một thứ tự tô-pô ẩn. Nếu một binh sĩ mới đẩy một binh sĩ cũ, người mới phải được thực hiện trước người cũ trong dãy NewGoto . Nếu hai khối người đứng cạnh nhau và khối trái có thể làm dịch khối phải, thì khối phải phải được đưa vào trước khối trái. Như vậy các ràng buộc “phải trước” tạo thành một đồ thị phụ thuộc, nhưng không nên xây toàn bộ đồ thị vì số cạnh có thể là bậc hai.

Cách trong bài viết là mô phỏng không hoàn chỉnh bằng các khối. Với mỗi khối, lưu một danh sách lệnh cmd . Khi chen một phần tử vào một khối, đưa lệnh của phần tử mới lên đầu danh sách của khối:

$$\text{cmd}(A + i) = (L_i, i) + \text{cmd}(A).$$

Khi hai khối A và B hợp nhất, nếu A nằm bên trái B , thì mọi phần tử của B phải xuất hiện trước mọi phần tử của A :

$$\text{cmd}(A + B) = \text{cmd}(B) + \text{cmd}(A).$$

Sau khi xử lý hết lệnh, nối các khối còn lại từ phải sang trái. Dãy thu được chính là thứ tự thực hiện NewGoto .

Để hiện thực, phần sinh thứ tự có thể dùng hợp nhất tập hợp và danh sách nối. Phần thực hiện NewGoto lại trở về bài tìm ô trống thứ k , giống ví dụ dựng cây. Nếu chiều dài cuối cùng của hàng là $N + M$, ta dùng cây Fenwick hoặc cây đoạn trên $N + M$ vị trí, đạt

$$\mathcal{O}(N \log(N + M)).$$

Trong một số biến thể, nếu thứ tự và vị trí có thể xử lý bằng danh sách tuyến tính sau khi đã biết toàn bộ lệnh, mốc $\mathcal{O}(N + M)$ trên slide cũng xuất hiện.

5 Kết luận

Hai ví dụ của slide đi theo cùng một khuôn: ban đầu quan hệ dữ liệu có vẻ rối, nhưng sau khi tìm đúng thứ tự, bài toán rút về các thao tác rất cơ bản. Với cây, thứ tự DFS biến số hậu duệ nhỏ hơn thành “ô

trống thứ $t(i) + 1$ ”. Với hàng binh sĩ, thứ tự tô-pô ẩn biến dãy chuyển đẩy của Goto thành dãy NewGoto không va chạm.

Thông điệp của bản trình chiếu là đừng vội tối ưu mô phỏng trực tiếp. Trước hết hãy hỏi quan hệ nào đang ép phần tử này phải đứng trước phần tử kia. Khi thứ tự ấy được lộ ra hoặc tự xây dựng được, thuật toán thường ngắn hơn, chứng minh rõ hơn, và độ phức tạp giảm từ bậc hai xuống gần tuyến tính hoặc $\mathcal{O}(N \log N)$.